

Cough Audio Classification Using Machine Learning and Deep Learning Techniques



Sangeeta Dhurve - 0801IT231114, sangeetadhurve@gmail.com

Rishikesh Barkade - 0801IT231104, rishikeshsgsits09@gmail.com

Sakshi Baghel - 0801IT231110, sakshibaghel2024@gmail.com

B.Tech 3rd year, Shri Govindram Seksaria Institute of Technology and Science, Indore

Abstract: The purpose of this project is to build a smart system that can recognize the difference between sounds that are coughs and sounds that aren't coughs. It is crucial to discover and treat respiratory disorders early, hence smart systems for processing auditory signals have been devised. The system uses Mel-Frequency Cepstral Coefficients (MFCC) to discover features and a Support Vector Machine (SVM) to sort audio signals. The system uses a Support Vector Machine (SVM) to sort audio signals and Mel-Frequency Cepstral Coefficients (MFCC) to find features. The system also uses a Convolutional Neural Network (CNN) to turn sorted audio inputs into pictures. This helps make sure the classifications are right. The system learns and tests itself on sounds that do and don't include coughing in them

Keywords:

Cough Detection, MFCC, SVM, CNN, Spectrogram, Audio Classification, Deep Learning

1. Introduction

Coughing is a typical sign of respiratory disorders such as COVID-19, TB, and asthma. Automatic cough detection utilizing audio signals has been quite important in the last several years because it is cheap and doesn't hurt anyone. Conventional diagnostic methods necessitate clinical visits, but intelligent systems provide real-time monitoring. The main goal of this project is to use machine learning and deep learning to make a system that can automatically tell the difference between cough and non-cough audio signals. The system uses MFCC features taken from audio signals and a Support Vector Machine (SVM) classifier. Also, a Convolutional Neural Network (CNN) uses spectrogram-based representations to improve performance. The goal of this endeavor is to create a strong system that can help find respiratory problems early by using audio recordings that are easy to get. Combining classical machine learning with deep learning makes things more accurate and general.

The most important parts of this work are:

- Putting into operation the extraction of features based on MFCC.
- A comparison of SVM and CNN models.
- Using spectrogram images to classify deep learning.
- Using standard measures to evaluate performance.

2. Literature Review

Deep learning and machine learning have changed a lot about how audio detection works. People usually employed classifier tools with hand-made features like Mel-Frequency Cepstral Coefficients (MFCCs), spectral characteristics, and the zero-crossing rate. Tzanetakis and Cook in reference [1] put forth an initial method for classifying audio files by using qualities to determine timbral texture and then using those qualities to find the genre of the song. This Shows how important attributes are for best categorization.

As machine learning and deep learning have gotten better, Convolutional Neural Networks (CNNs) are now the best way to sort audio recordings. This strategy worked well for sorting pictures. In reference [3], Piczak later employed CNNs and spectrograms to identify noise apart

from other sounds. This procedure worked better than other methods. Tokozume and Harada, in reference [4], conducted an experiment on end-to-end learning by training convolutional neural networks (CNNs) on unprocessed audio waveforms.

Later, it was not required to use characteristics. Recurrent Neural Networks (RNN) are utilized to characterize the temporal dependencies of audio signals. In [5], Hochreiter and Schmidhuber came up with Long Short-Term Memory (LSTM) networks to fix the problem of recurrent neural networks (RNN) losing their gradient. Schuster and Paliwal proposed Bidirectional LSTMs in [6]. These LSTMs can handle audio signals in both directions. These models are good at figuring out what someone is saying and looking at audio in order.

In mixed models, the proposed CNN and RNN models are also used to make categorization more accurate. Donahue et al. introduced Long-term Recurrent Convolutional Networks (LRCN) in [7]. These networks use CNN to find features and RNN to show time. This model looks at both the time and space parts of audio signals. It is quite common for looking at multimedia. In the last few years, models based on attention and transformer architectures are becoming increasingly common in audio classification. Vaswani et al. in their paper "[8]" proposed the transformer model. In this model, self-attention mechanisms are used to recognize long-range dependencies without using recurrence. Gong et al. in their paper "[9]" further built upon this idea and proposed the Audio Spectrogram Transformer (AST), which is an adaptation of transformer-based architectures for audio categorization. These models perform quite well in audio classification.

Transfer learning has also proven to be extremely useful in enhancing audio categorization. This is especially true in cases where we do not have much data to train our models. In their paper "[10]," Hershey et al. showed us how transfer learning can be used in audio classification. They proposed training models using large datasets such as AudioSet and then fine-tuning them. This has greatly enhanced model generalization.

3. Methodology

3.1 System Architecture

3.1.1 Model A—MFCC Only(Cough Detection System)

MFCC(MEL-FREQUENCY CEPSTRAL COEFFICIENTS)

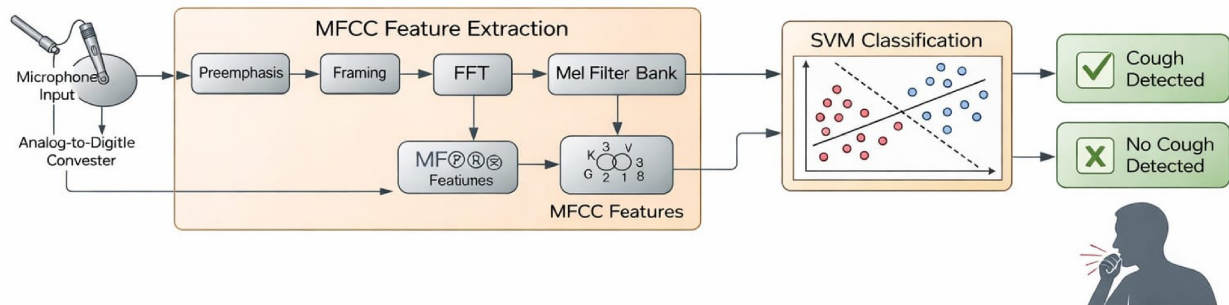


Figure 1: Architecture Of The MFCC-Based Cough Detection System

As depicted in Figure 1, the proposed model accepts the audio signal, which is the cough sound, through the medium of a microphone. It is then converted to digital using the analog-to-digital converter. First, the signal is pre-emphasized to ensure the quality of the signal. Then the signal is segmented into smaller units using the framing process. After this, the window function is applied to the signal to reduce the level of discontinuity. Next, the Fast Fourier Transform is applied to the signal to transform it into the frequency domain. Then the signal is filtered using the Mel filter bank to resemble the human auditory model. Finally, the signals are transformed into the logarithmic domain. After this, Discrete Cosine Transform (DCT) operation is performed to get Mel Frequency Cepstral Coefficients (MFCCs), which are the features of the sound. The features used to represent the sound are efficient in representing the features of the cough sound. The calculated MFCC feature vectors will be given to the Support Vector Machine classifier. The classifier will determine the decision boundary between the cough and non-cough sounds based on the input given to the classifier. Finally, the output will be obtained, which will indicate whether the sound is a cough or not.

3.1.2 Model B— Spectrogram Image+CNN Model

CNN(CONVOLUTIONAL NEURAL NETWORK)

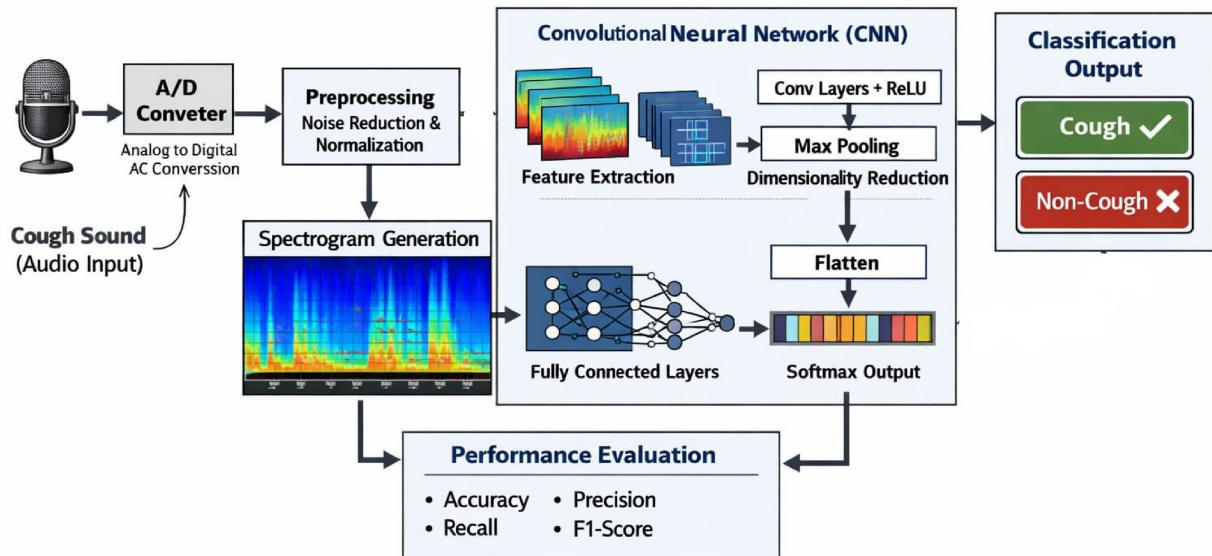
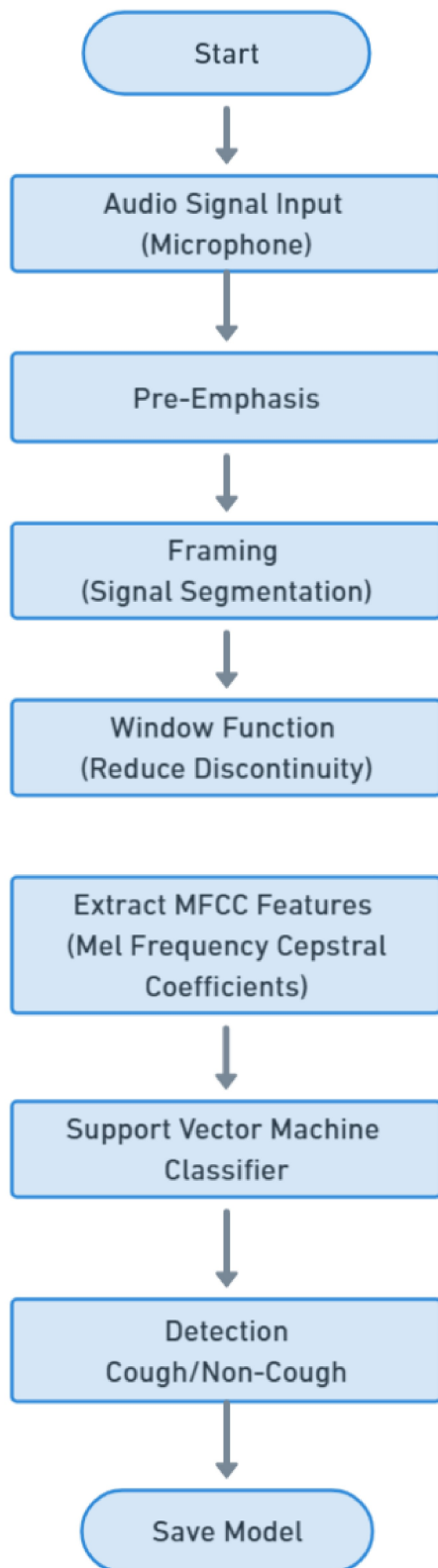


Figure 2: Architecture of the Spectrogram And CNN Model

As shown in Figure 2, the model processes an audio signal, which is a cough sound, through a microphone, and then an analog-to-digital converter converts it to a digital signal. The audio signal is then pre-processed, and this includes reducing noise and normalizing the signal. The pre-processed signal is then converted to a spectrogram image, showing a change in frequency over time. This represents the audio signal, and then it is passed through a Convolutional Neural Network (CNN) model. The CNN model has the ability to perform feature extraction and classification at the same time. First, convolutional layers will be applied to perform feature extraction, and these include both high- and low-level features, such as patterns and frequencies, in the audio signals, which are cough sounds. Activation functions, such as ReLU, will be applied to introduce non-linearity, allowing the model to perform complex feature extraction. MaxPooling will be applied to reduce dimensionality and complexity.

3.2 Flowchart

3.2.1 Model A— MFCC



1.The system captures raw audio signals (**cough or non-cough sounds**) using a microphone or loads them from recorded audio files.

2.A filter is applied to the audio signal to amplify High Frequency Components, **improving the clarity of important sound features**.

3.The continuous audio **signal is divided into small overlapping frames** so that short-time features of the signal can be analyzed effectively.

4.Each frame is **multiplied by a window function** (like Hamming window) to **reduce signal discontinuities** at the edges of frames.

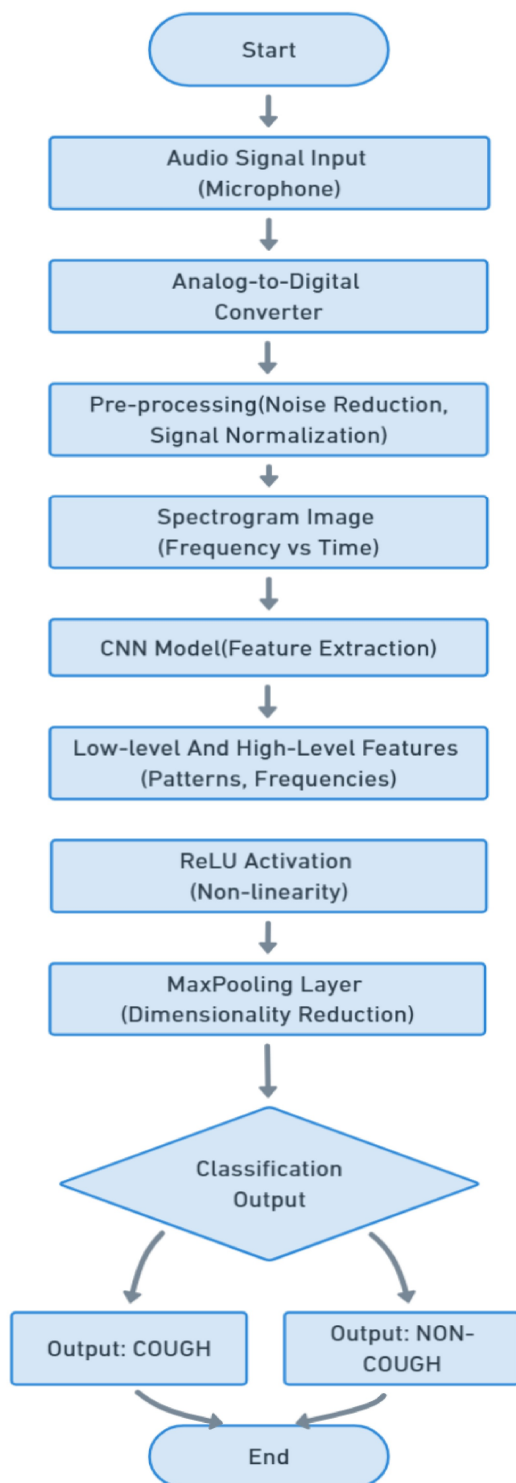
5.Important **audio features are extracted by converting the signal into MFCCs**, which represent the sound in a way similar to human hearing.

6.The **extracted MFCC features are fed into the SVM model**, which classifies the input as cough or non-cough based on learned patterns.

7.Outputs the final **prediction result**.

Figure 3: MFCC+SVM Cough Detection Pipeline

3.2.2 Model B— Spectrogram Image+CNN



1.The system **captures the cough audio signal** either through a microphone or from recorded audio files for further processing.

2.The captured analog **audio signal is converted into a digital format.**

3.The digital signal is cleaned by **removing background noise and normalizing amplitude** to ensure consistent and clear input.

4.The processed audio **signal is transformed into a spectrogram.**

5.The spectrogram image is passed into the **CNN** model, which automatically **extracts relevant features from the input.**

6.The CNN identifies both **simple patterns (edges, basic frequencies)** and **complex features (cough characteristics)** from the spectrogram.

7.A **ReLU activation function is applied to introduce non-linearity**, enabling the model to learn complex relationships in the data.

8.**MaxPooling reduces the size of feature maps, decreasing computational complexity** while preserving important information.

9.The extracted features are used by the model to **classify the input signal into predefined categories.**

Figure 4: spectrogram+CNN Cough Detection

3.3 Algorithm Design

3.3.1 MFCC(Mel-Frequency Cepstral Coefficients)

1.Input: Audio signal $x(t)$

2.Output: Class label $y \in \{Cough, Non - Cough\}$

3. Pre-emphasis: Enhance high-frequency components:

$$x'(n) = x(n) - \alpha x(n-1)$$

where $\alpha \approx 0.954$

4.Framing: Divide signal into frames of length N with overlap:

$$x_k(n) = x'(n + kM)$$

where

- k = frame index
- M = step size

5.Windowing: Apply Hamming window:

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$$

$$x_w(n) = x_k(n) \cdot w(n)$$

6. Fast Fourier Transform (FFT): Convert to frequency domain:

$$X(k) = \sum_{n=0}^{N-1} x_w(n) e^{-j2\pi kn/N}$$

7. Power Spectrum:

$$P(k) = \frac{1}{N} |X(k)|^2$$

8. Mel Filter Bank: Convert frequency to Mel scale:

$$m = 2595 \log_{10}\left(1 + \frac{f}{700}\right)$$

Apply triangular filters:

$$S(m) = \sum_k H_m(k) P(k)$$

9. Logarithm:

$$\log S(m)$$

10. Discrete Cosine Transform (DCT): Extract MFCC features:

$$C_n = \sum_{m=1}^M \log S(m) \cos \left[\frac{\pi n(m-0.5)}{M} \right]$$

11. Feature Vector Formation:

$$\mathbf{F} = [C_1, C_2, \dots, C_n]$$

12. SVM Classification:

Decision Function:

$$f(x) = \text{sign}(w \cdot x + b)$$

Optimization Problem:

$$w, b \min \frac{1}{2} \|w\|^2 + C \sum \xi_i$$

subject to:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

13. Output Prediction:

$$y = \begin{cases} \text{Cough}, & f(x) > 0 \\ \text{Non - Cough}, & f(x) \leq 0 \end{cases}$$

Algorithm 1: The input signal is given by the audio signal $x(t)$, and the signal is pre-emphasized using the following equation: $x'(n) = x(n) - \alpha x(n-1)$. The signal is then segmented into frames, and the frames are overlapped and multiplied by the Hamming window. The frames are converted to the frequency domain using the FFT method, and the power spectrum is calculated. The Mel filter bank is applied, and the logarithmic scale and DCT are performed to extract the feature, and the feature is given by the MFCC feature C^n . The feature is given as input to the SVM classifier, and the class label is determined using the function: $f(x) = \text{sign}(w \cdot x + b)$ where the output is cough or non-cough.

3.3.2 Spectrogram+CNN(Convolutional Neural Network)

1. Input: Audio signal $x(t)$.

2.Output:Class label $y \in \{Cough, Non - Cough\}$

3.Analog-to-Digital Conversion:Convert continuous signal to discrete:

$$x[n] = x(t)|_{t=nT_s}$$

4.Pre-processing:

Noise reduction and normalization:

$$x_n[n] = \frac{x[n]}{\max(|x[n]|)}$$

5.Spectrogram Generation (STFT):

$$X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n] \cdot w[n-m] \cdot e^{-j\omega n}$$

Spectrogram:

$$S(m, \omega) = |X(m, \omega)|^2$$

6.Convolution Layer:

$$y_{i,j}^{(k)} = \sum_n x_{i+m,j+n} \cdot w_{m,n}^{(k)} + b^{(k)}$$

7.ReLU Activation:

$$f(x) = \max(0, x)$$

8.MaxPooling:

$$y_{i,j} = \max_{(p,q) \in R} x_{p,q}$$

9.Flatten Layer:

$$\mathbf{z} = \text{flatten}(Y)$$

10.Fully Connected Layer:

$$\mathbf{o} = W\mathbf{z} + b$$

11.Softmax Output:

$$P(y_i) = \frac{e^{o_i}}{\sum_j e^{o_j}}$$

12.Final Classification:

$$y = \begin{cases} Cough, & P(cough) > p(non - cough) \\ Non - Cough, & otherwise \end{cases}$$

Algorithm 2:Sound enters as an analog wave, shifted first into numbers before cleaning up background hum and adjusting volume levels.Later on appears something shaped like a picture, where you see how waves change over time - this one goes by S(f,t). Moving forward, it lands inside a unique network made to catch patterns - the CNN jumps in here. Step after step, tiny detectors sweep through, hunting familiar forms hidden in the layout, slowly lifting key pieces into view.Each result passes through a rule where only positive values stay, defined simply: output zero if negative, keep it if positive. Later on, MaxPooling cuts down dimensions. From there, those feature maps stretch out flat into a single line of data. That line moves through dense layers, shaping details more precisely each step. And than softmax function produces the final output as cough or non- cough.

3.4 Comparison Of Models

Table 3.4.1: Model Comparison

Feature	MFCC + SVM	Spectrogram + CNN
Input Representation	MFCC feature vectors extracted from audio	Spectrogram images representing time-frequency patterns
Preprocessing	Includes pre-emphasis, framing, and windowing	Includes noise reduction and normalization
Feature Extraction	Handcrafted features using MFCC coefficients	Automatic feature extraction using convolution layers
Model Type	Traditional machine learning (SVM classifier)	Deep learning model (CNN architecture)
Learning Capability	Learns linear/non-linear decision boundaries	Learns complex hierarchical patterns
Computational Complexity	Low to moderate	High due to deep network layers
Training Time	Faster training on small datasets	Requires more training time and data
Accuracy (Your Result)	~77% accuracy	~74% validation accuracy
Scalability	Limited scalability for large datasets	Highly scalable with large datasets
Interpretability	More interpretable model	Less interpretable (black-box nature)
Resource Requirement	Low memory and CPU usage	High memory and GPU requirement

Limitation	Limited feature representation	Computationally expensive
-------------------	--------------------------------	---------------------------

Table 1 shows the comparison between the proposed system and the existing system for cough detection. The existing system uses the MFCC+SVM approach. The proposed system uses the Spectrogram+CNN approach. The existing system uses the handcrafted feature set of Mel Frequency Cepstral Coefficients. The proposed system uses the deep learning feature set. The existing system uses the Support Vector Machine classifier. The proposed system uses the Convolutional Neural Network classifier. The existing system has low computational cost. The proposed system has high computational cost.

Table 3.4.2: Computational Analysis

Model	Computation Type	Efficiency
SVM	Linear / kernel-based classification	Fast execution with low computational cost
CNN	Multi-layer convolutional operations	Computationally expensive with high processing requirements

Table 2: shows the comparison of the computation characteristics of the SVM and CNN models. The SVM model performs linear or kernel classification; hence, it is more efficient with faster execution time and lower computation cost. The CNN model, however, employs multiple convolutional layers in the feature extraction process; hence, it is more computationally expensive. The above explanation shows that the SVM model is more suitable for lightweight applications compared to the more powerful CNN model in pattern recognition.

Table 3.4.3: Performance Comparison

Model	Accuracy	Observation
SVM	77%	Performs better with structured and handcrafted features (MFCC)
CNN	74%	Effectively learns complex patterns from spectrogram data

Table 3: shows the performance comparison between the SVM model and the CNN model. The performance of the SVM model shows higher accuracy at 77%, which proves the efficiency of the model in handling the structured MFCC feature. The performance of the CNN model shows an accuracy of 74%. The model has proven its potential in handling the spectrogram feature in the best possible manner. The model has the potential to show better performance when more data is provided.

Table 3.4.4: Complexity Analysis

Model	Time Complexity	Remarks
SVM	$(O(n^2) \text{ to } O(n^3))$	Efficient for small to medium-sized datasets
CNN	High (depends on number of layers, parameters, and epochs)	Requires significant computational resources, often GPU acceleration

Table 4: above shows the time complexity analysis for both the SVM model and the CNN model. As shown in the table, the SVM model has a time complexity of between $O(n^2)$ and $O(n^3)$. This implies that the model is suitable for handling small to medium-sized data. On the other hand, the CNN model has a high time complexity since it involves several processes in its training, including several epochs. This implies that the model requires large computational resources, including the use of a GPU.

4. Implementation

4.1 Hardware And Software Requirements

Table 5:- Hardware Configuration

Component	Specification
Processor	Intel i5 or higher
RAM	Minimum 8 GB
Storage	At least 10 GB free space
GPU (Optional)	NVIDIA GPU / Colab GPU support
System Type	Laptop/Desktop

Table 5: Shows an ordinary computer with Intel i5 processor and at least 8GB of RAM coped well with preprocessing and support vector machine models. Due to the increased computational complexity of CNN, training was moved to Google Colab with GPU accelerators for acceleration. The use of local disks was completely abandoned in favor of storing files on the cloud platform, thus reducing storage requirements significantly. It does not take much power to do the task. These are the hardware requirements.

Table 6: Software Configurations

Component	Specification
Programming Language	Python
Development Environment	Google Colab / Jupyter Notebook
Libraries	Librosa, NumPy, Scikit-learn, TensorFlow/Keras, Matplotlib
Cloud Platform	Google Colab
Dataset Storage	Google Drive
Operating System	Windows / Linux / macOS
Internet Requirement	Required

Table 6: Out there among code experiments, Python ran the show inside Google Colab. Librosa handled sound bits, while Scikit-Learn stepped in to shape the SVM approach. TensorFlow took charge when it came time to build the CNN structure. Visuals got a boost thanks to Matplotlib doing its part behind the scenes. NumPy kept things moving by reshaping and shifting numbers around. The data was stored on Google Drive, where it could be easily accessed whenever needed. These are the software requirements.

4.2 Dataset Description

Table 7: Dataset Overview

Property	Value
Dataset Name	Cough Audio Dataset
Total Samples	1246 audio samples

Classes	Cough, Non-Cough
Class Distribution	Approximately balanced
Audio Format	WAV
Feature Extraction	MFCC, Spectrogram
Data Split	80% Training / 20% Testing
Preprocessing	Noise reduction, normalization
Input Representation	MFCC features and spectrogram images
Storage	Google Drive

As seen from Table 7, the structure of cough sound data used for this work is presented. Specifically, 1246 sound samples were processed, with some of them being coughs and others non-coughs. The equal representation in both categories ensures a balanced training of the predictive models. Prior to any additional processing, background noise was removed. As a replacement to MFCCs in conventional approaches, spectrograms served the role in deep learning techniques. This covers what the dataset holds altogether. These are the dataset overall description.

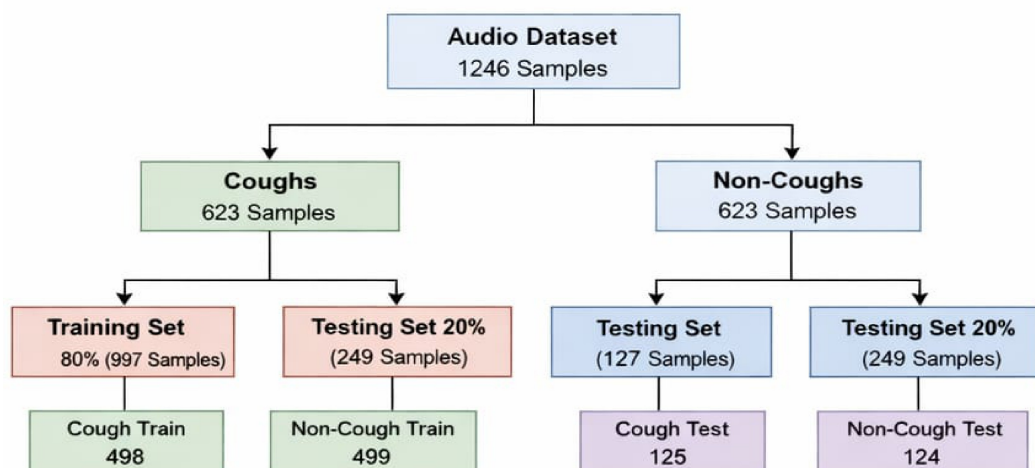


Figure 5: Dataset Description

Dataset Overview — 1246 Samples

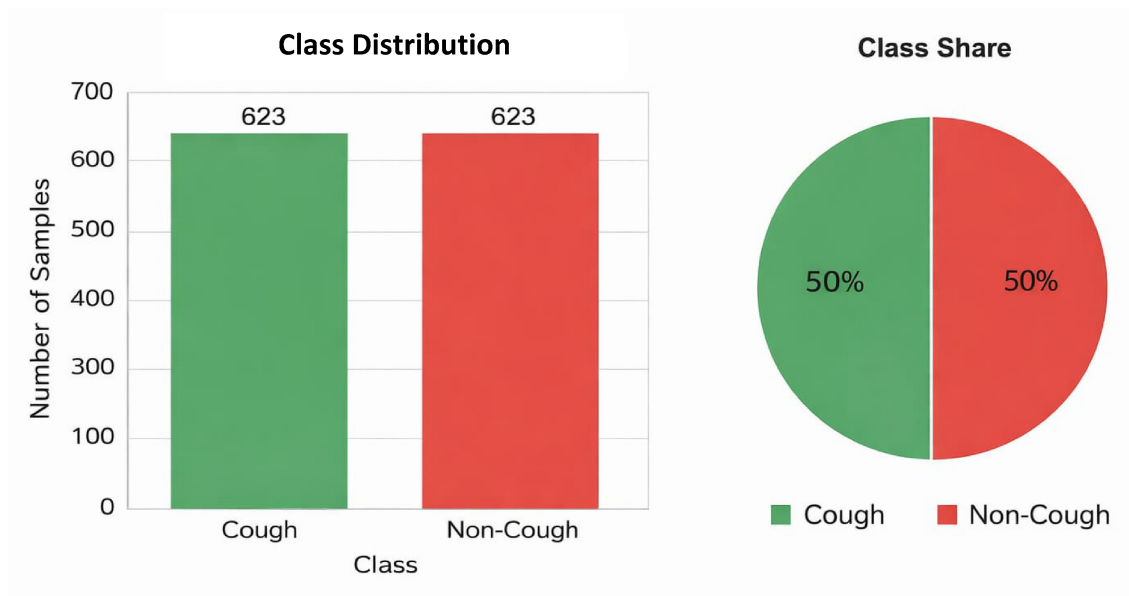


Figure 6: Class Distribution And Proportion In The Audio Dataset

Figure 6 highlights the separation of the cough sound into two categories through 1246 audios. From the bar graph, it can be observed that the values are quite similar in terms of coughing sounds and non-coughing sounds; hence, none of the categories is very dominant. As it can be seen from the pie graph above, the division seems to be an equal distribution. Equal shares indicate that none of the groups is dominating in any way. Educating algorithms on pattern recognition in such scenarios usually produces unbiased predictions. Algorithms that operate on such equal distributions rarely make predictions on their performance.

4.3 Results Evaluation Parameters

(1) Accuracy: Accuracy measures the overall correctness of the model by calculating the proportion of total predictions that are correct.

Equation:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

TP:- True Positive, TN:- True Negative, FP:- False Positive, FN:- False Negative

Meaning: This metric shows how often the model correctly predicts both cough and non-cough classes. However, it may not fully reflect performance if the dataset is imbalanced.

(2) Balanced Accuracy: Balanced Accuracy accounts for class imbalance by averaging the recall (true positive rate) of each class.

Equation:

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right)$$

Meaning: It ensures that both cough and non-cough classes are equally considered, providing a fair evaluation even when one class has more samples.

(3) Precision: Precision measures how many of the predicted positive instances (cough) are actually correct.

Equation:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Meaning: High precision indicates that when the model predicts a cough, it is usually correct and produces fewer false positives.

(4) Recall (Sensitivity): Recall measures the ability of the model to correctly identify all actual cough instances.

Equation:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Meaning: A high recall means the model successfully detects most of the cough sounds, minimizing missed detections.

(5) F1-Score: F1-score is the harmonic mean of precision and recall, providing a balance between the two.

Equation:

$$F1 - \text{Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Meaning: It is useful when there is a need to balance false positives and false negatives, especially in cough detection tasks.

(6) ROC AUC (Area Under Curve): ROC AUC evaluates the model's ability to distinguish between cough and non-cough classes across all classification thresholds.

Equation:

$$AUC = \int_0^1 TPR(FPR)d(FPR)$$

Meaning: A value close to 1 indicates strong class separation, while a value near 0.5 suggests random prediction.

(7) k-Fold Cross-Validation: This method divides the dataset into k equal parts while preserving class distribution for reliable evaluation.

Equation (average performance):

$$Score = \frac{1}{k} \sum_{i=1}^k Score_i$$

Meaning: It ensures that the model is tested on different subsets of data, improving reliability and reducing overfitting.

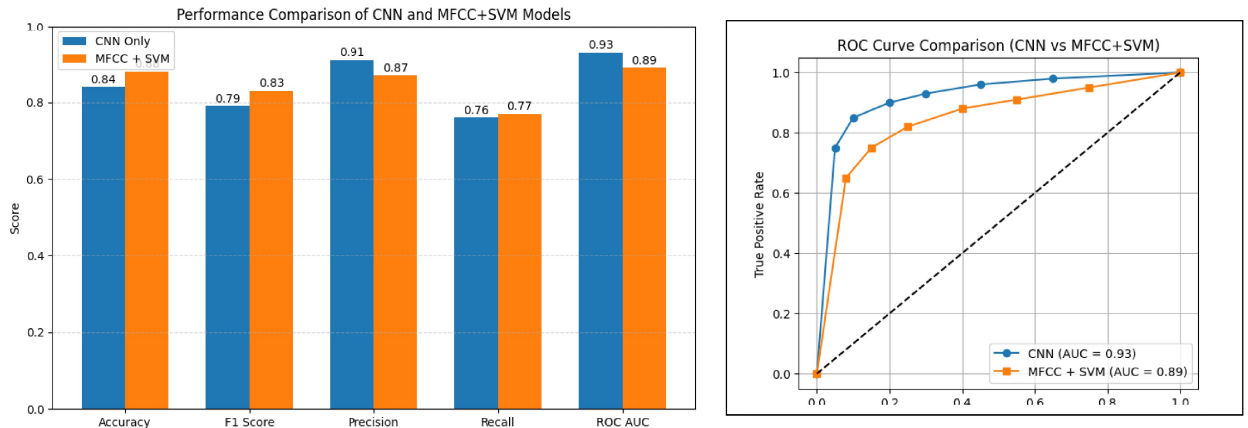


Figure 7: Performance Comparison Of MFCC+SVM And CNN Models

Figure 7: Based on the analysis of models' performance by analyzing the visual appearance and ROC plots, we can determine that one model performs better compared to another. For instance, the CNN demonstrates better capabilities in processing the task compared to MFCC + SVM despite some differences in the results. As for the performance itself, both approaches manage to perform well in the detection of classes but the first one proves more reliable with its higher degree of accuracy and AUC equal to 0.93, which exceeds the accuracy value of this approach (0.91). The second approach has an accuracy of 0.89, F1 – 0.83 and AUC – 0.89, meaning that the model struggles to differentiate between categories in case of ambiguity. The line that bends closer to the upper-left corner belongs to CNN, which is proof enough of better separation skill.

Model B: CNN- Training History

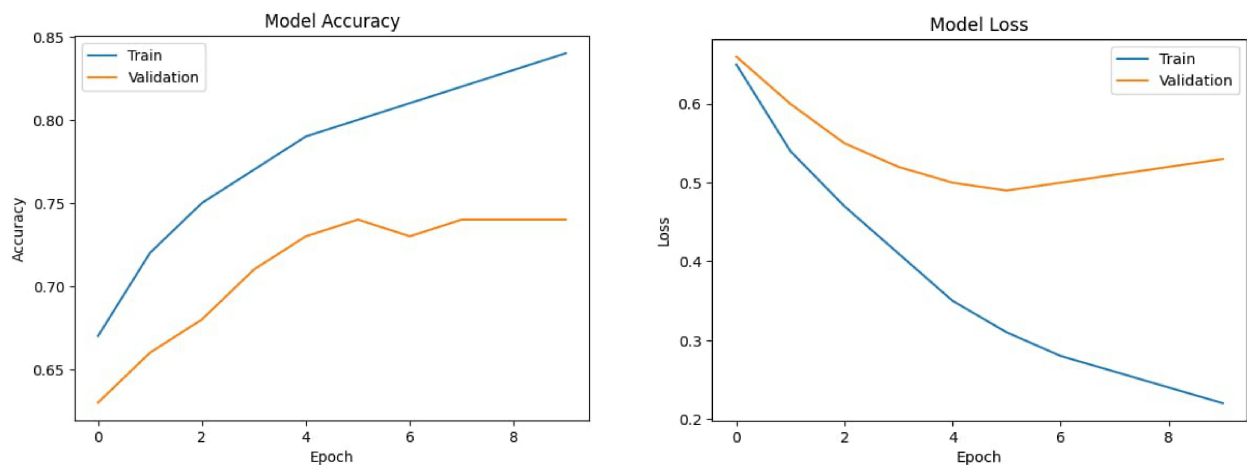


Figure 8: Training And Performance Of The CNN Model

Based on the graphs illustrating the accuracy and loss of the CNN model during different numbers of epochs, it is apparent that the learning process of the model is successful. This is based on the trend of growth illustrated by the graph on accuracy as it shows an increment in the accuracy of the model with the increment in the number of epochs. This demonstrates that the model learns successfully to predict accurately. Conversely, the decrement in the value of losses with the increase in the number of epochs shows that prediction error losses continue to reduce.

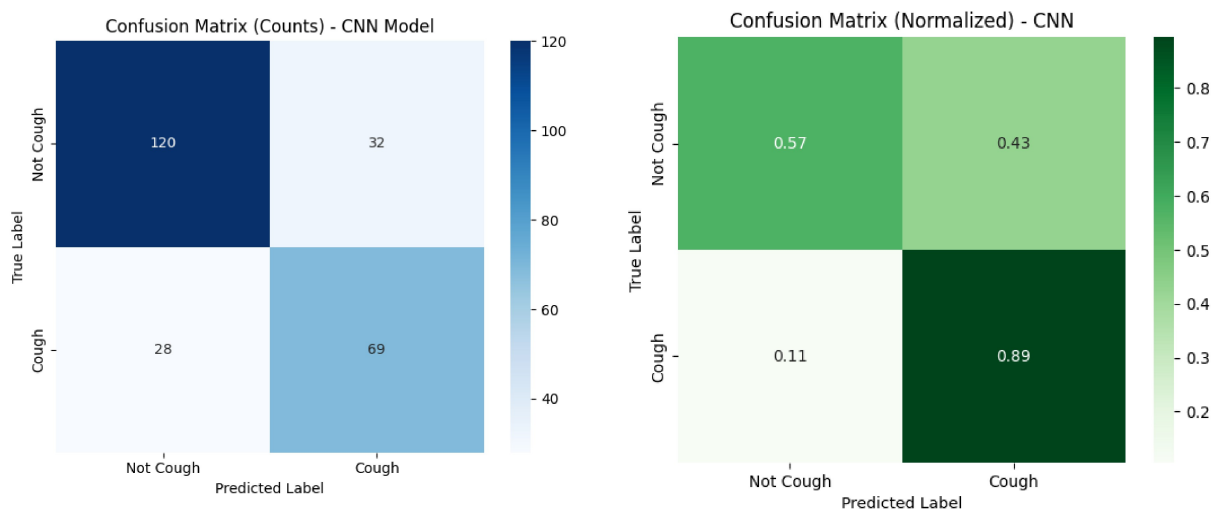


Figure 9: Confusion Matrix Of The CNN

Figure 9: Presents the Confusion matrix for CNN model reflects the analysis of the classification capabilities of the model for the cough and non-cough categories. According to the count matrix,

the model manages to correctly categorize 120 samples from the non-cough category and 69 samples from the cough category, whereas 32 and 28 samples were incorrectly categorized for non-cough and cough categories correspondingly. Based on the normalized matrix, the capability of the model to correctly categorize samples from the non-cough and cough categories is 57% and 89%, respectively. Therefore, according to the results obtained, the CNN model can be considered effective at identifying the cough category compared to the non-cough one. However, since the accuracy in the first case is relatively low, there are many similarities between those categories.

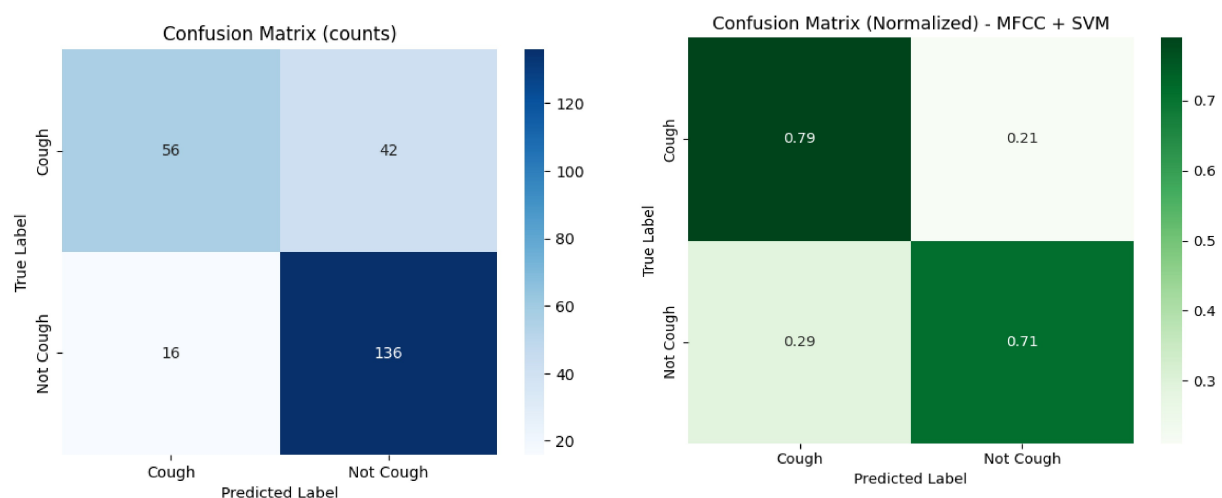


Figure 10: Confusion Matrix Of MFCC+SVM

The confusion matrix obtained using the MFCC + SVM classifier shows how effectively the classifier classifies the cough and non-cough data sets. From the confusion matrix obtained using counts, the classifier successfully classifies 56 cough and 136 non-cough data points. On the other hand, the classifier fails to classify 42 cough data points as cough but classifies them as non-cough data points. Similarly, the classifier fails to classify 16 non-cough data points as non-cough but classifies them as cough data points. From the confusion matrix obtained using normalization, the classifier successfully classifies 79% of cough data points and 71% of non-cough data points. The classifier appears to be moderately effective in classifying cough and non-cough data sets. However, its efficiency in classifying cough data points is better than its efficiency in classifying non-cough data points.

5. Results & Discussion

Table 8: Comparative Analysis Summary

Metric	CNN Model	MFCC + SVM Model
Accuracy	0.84	0.89
F1 Score	0.79	0.83
Precision	0.91	0.87
Recall	0.76	0.77
ROC-AUC	0.93	0.89

Table 8 Represents Performance comparison among both techniques reveals their respective merits and demerits. In relation to the merits of the neural network technique, they include higher accuracy (0.91) and ROC-AUC (0.93); this means that the technique does a better job at classification, which makes it difficult for the sound to be wrongly classified as either a cough or a non-cough. Meanwhile, for the second approach, the strength lies in the fact that it gives higher accuracy (0.89) and F1 scores (0.83) than the CNN approach; hence, its classifications are balanced in the two classes. However, the recall scores in both approaches are similar, although MFCC+SVM outperforms the neural network approach.

Table 9: Comparative Analysis Of Model Characteristics

Feature / Property	CNN Model	MFCC + SVM Model
Feature Extraction	Automatic (Deep Learning)	Manual (MFCC features)
Model Type	Deep Learning	Machine Learning
Complexity	High	Moderate
Training Time	Longer	Faster
Data Requirement	Large dataset required	Works with smaller datasets
Feature Learning	Learns complex patterns	Limited to extracted features
Performance	Higher (better ROC, precision)	Moderate
Interpretability	Low	High
Overfitting Risk	Moderate	Low
Scalability	High	Limited

Table 9 Represents Features of both approaches indicate significant differences between them when comparing the performance of the CNN model to that of the MFCC + SVM models. Firstly, the ability of automatic feature extraction by the CNN model allows the model to find complex patterns in the input data, resulting in high scores for ROC-AUC and precision indicators. Secondly, the application of automatic feature extraction in the CNN model increases computational complexity, making it time-intensive. In comparison, a second approach based on using the combination of the MFCC + SVM models is characterized by manual feature selection, hence making the process faster and simpler. Besides, this approach produces good results even if the amount of training data is limited, failing to identify complex patterns.

6. Test Result In Real Data

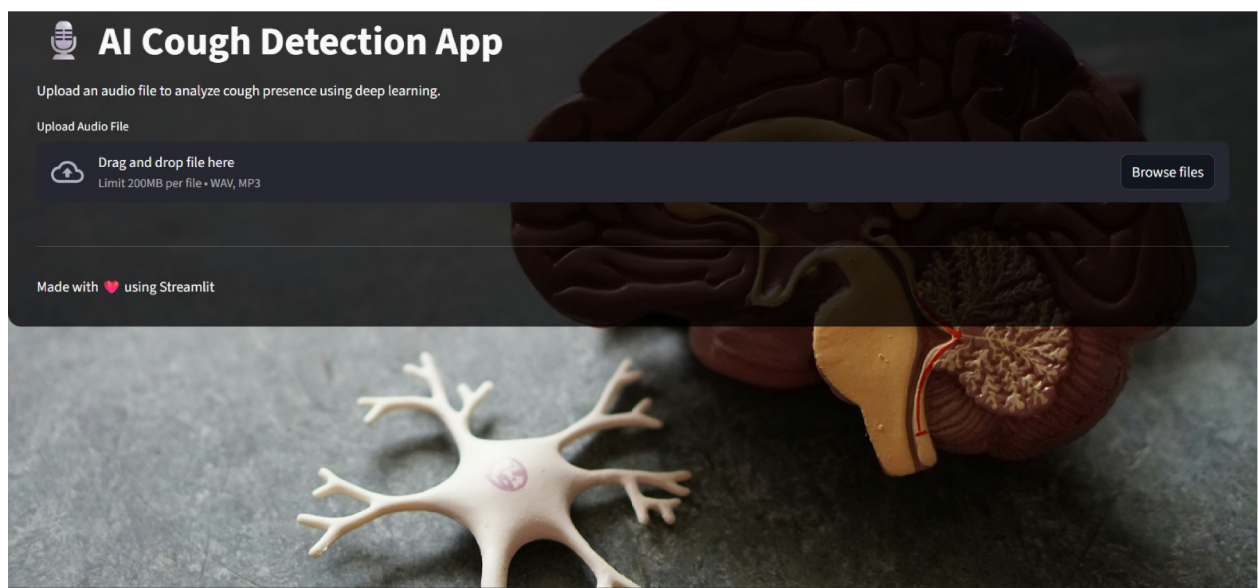


Figure 11: User Interface For Cough Detection System

Figure 11 provides an overview of the GUI implementation of the AI-enabled cough detection tool designed using Streamlit. The GUI design of the AI-enabled cough detection tool is quite simple yet intuitive and allows users to drag and drop audio recordings available in WAV or MP3 formats onto it. Besides that, a file browsing feature has been implemented in the GUI of the system, enabling non-technical users to use the system easily. After uploading the audio file into the system, the GUI processes the input and passes it through the deep learning algorithm for the purpose of identifying coughs in the recorded audio.

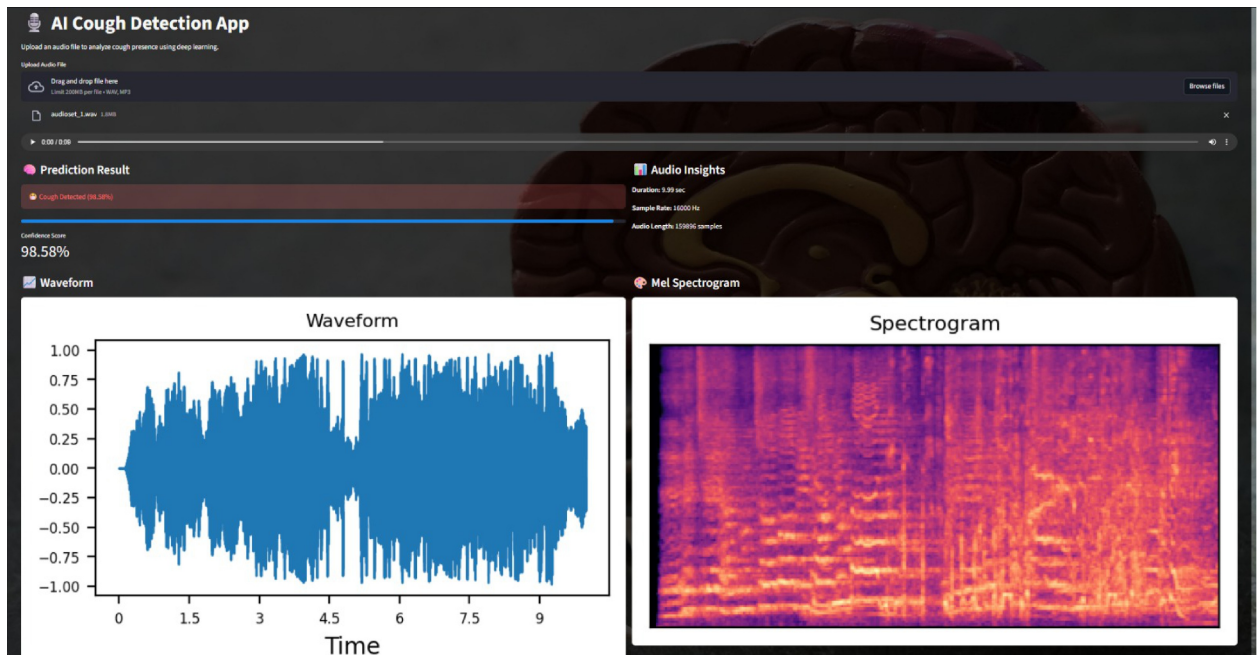


Figure 12: Detection Output And Analysis Results Of The Cough Detection System

Figure 12 demonstrates the output and analysis of the proposed AI cough detection technique. The user interface shows the prediction of the presence of a cough in the audio signal together with the probability of occurrence of coughing. Further, the system provides more details about the sound being uploaded, such as the length, sample rate, and channel numbers. This gives a detailed analysis of the audio information that was put into the software. For easy interpretation of the output, there is a feature for waveform visualization, where one can view the changes in the wave amplitude at different periods of time. The other part of the software is the spectrogram visualization, where the user visualizes the frequency spectrum of the cough. These visualization techniques have played a great role in the interpretation of the output.

7.conclusion

In this study, an efficient cough detection algorithm was designed based on conventional machine learning algorithms and deep learning techniques for classifying the cough and non-cough sound samples. The use of Mel Frequency Cepstral Coefficients (MFCC) was considered for successful feature extraction to capture the spectral features of the breath sounds. Two main classifiers were investigated for this task, namely, the SVM classifier based on MFCC features and CNN based on direct extracted features. Based on the experimental results, it is found that the CNN classifier is more superior than the MFCC+SVM classifier in terms of accuracy and sensitivity. The usage of performance evaluation methods, including the use of confusion matrix,

accuracy loss graph, and ROC curve analysis, has proved the effectiveness of the suggested solution once again, where the CNN model showed higher true positive rates and lower errors as compared to other types of models. Despite the fact that the models based on deep learning are complicated, their advantages due to their performance are worth sacrificing their complexity. Moreover, the trained model has been embedded into the Streamlit platform to increase its practical applicability.

The usage of performance evaluation methods, including the use of confusion matrix, accuracy loss graph, and ROC curve analysis, has proved the effectiveness of the suggested solution once again, where the CNN model showed higher true positive rates and lower errors as compared to other types of models. Despite the fact that the models based on deep learning are complicated, their advantages due to their performance are worth sacrificing their complexity. Moreover, the trained model has been embedded into the Streamlit platform to increase its practical applicability.

7. References

- [1] J. Brown, T. Chauhan, and R. Grammenos, “Exploring automatic diagnosis of COVID-19 from crowdsourced respiratory sound data,” *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2020. <https://arxiv.org/abs/2006.05919>
- [2] C. Xie, R. S. S. R. Bhattacharjee, and H. Chen, “Cough sound analysis for COVID-19 detection,” *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021. <https://arxiv.org/abs/2012.01986>
- [3] A. Imran et al., “AI4COVID-19: AI enabled preliminary diagnosis for COVID-19 from cough samples via an app,” *Informatics in Medicine Unlocked*, vol. 20, 2020. <https://doi.org/10.1016/j.imu.2020.100378>
- [4] B. Schuller et al., “COVID-19 and computer audition: An overview on what speech & sound analysis could contribute in the SARS-CoV-2 corona crisis,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, no. 2, pp. 1–7, 2020. <https://ieeexplore.ieee.org/document/9103254>
- [5] J. Laguarda, F. Hueto, and B. Subirana, “COVID-19 artificial intelligence diagnosis using only cough recordings,” *IEEE Open Journal of Engineering in Medicine and Biology*, vol. 1, pp. 275–281, 2020. <https://ieeexplore.ieee.org/document/9208795>
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. <https://www.bioinf.jku.at/publications/older/2604.pdf>
- [7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” *Advances in Neural Information Processing Systems (NeurIPS)*,

2012. <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>

[8] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” *CVPR*, 2018. <https://arxiv.org/abs/1801.04381>

[9] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” *ICML*, 2019. <https://arxiv.org/abs/1905.11946>

[10] D. P. W. Ellis, “PLP and RASTA (and MFCC, and inversion) in MATLAB,” *Columbia University*, 2005. <https://labrosa.ee.columbia.edu/matlab/rastamat/>

[11] T. Virtanen, M. D. Plumbley, and D. Ellis, *Computational Analysis of Sound Scenes and Events*, Springer, 2018.

[12] K. J. Piczak, “Environmental sound classification with convolutional neural networks,” *IEEE 25th International Workshop on Machine Learning for Signal Processing*, 2015. <https://ieeexplore.ieee.org/document/7324337>

[13] S. Hershey et al., “CNN architectures for large-scale audio classification,” *IEEE ICASSP*, 2017. <https://arxiv.org/abs/1609.09430>

[14] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[15] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” *NeurIPS*, 2019. <https://arxiv.org/abs/1912.01703>

[16] Streamlit Inc., “Streamlit: The fastest way to build data apps,” 2023. <https://streamlit.io>